INFORMATIONSBLATT

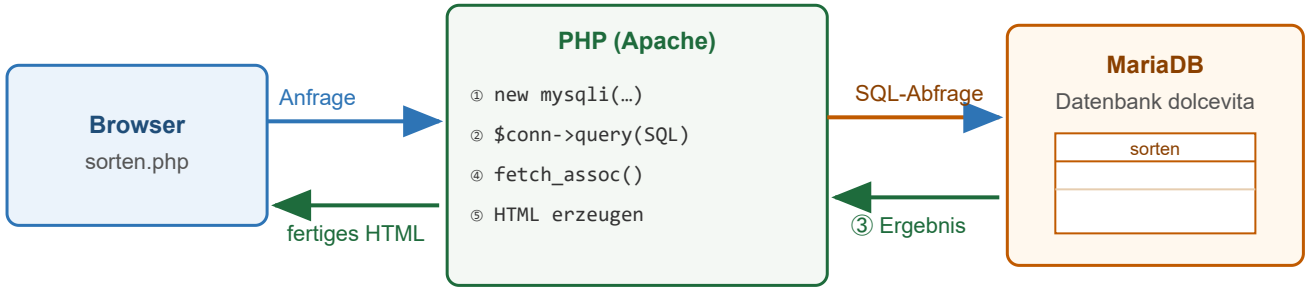
IB02 – PHP liest aus der Datenbank

Dynamische Webseiten mit PHP · LS08 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie verbinden PHP mit MariaDB, führen eine SQL-Abfrage aus, durchlaufen das Ergebnis Zeile für Zeile und reagieren sinnvoll, wenn die Datenbank nicht erreichbar ist.

1 | Wer spricht mit wem?



Der Browser spricht nie direkt mit der Datenbank – nur PHP tut das.

Abbildung 1: PHP ist der Vermittler zwischen Browser und Datenbank.

2 | Die Verbindung

PHP spricht über die Erweiterung **mysqli** mit MariaDB. `new mysqli(…)` baut die Verbindung auf und liefert ein Verbindungsobjekt, das man üblicherweise `$conn` nennt. Befehle an die Datenbank schickt man dann mit dem Pfeil `->`: `$conn->query(…)`.

verbindungstest.php verbindungstest.php

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Verbindung</title></head>
<body>

<?php
  $conn = new mysqli("localhost", "root", "", "dolcevita");
  $conn->set_charset("utf8mb4");
  echo "<p>Verbindung steht.</p>";

  $ergebnis = $conn->query("SELECT COUNT(*) AS anzahl FROM sorten");
  $zeile = $ergebnis->fetch_assoc();
  echo "<p>In der Tabelle stehen " . $zeile["anzahl"] . " Sorten.</p>";
?>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS

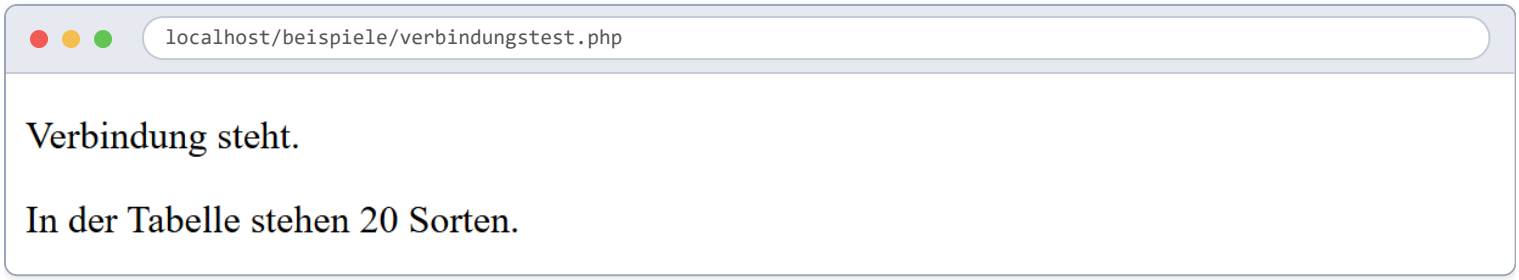


Abbildung 2: Verbindung aufgebaut, erste Abfrage ausgeführt.

Parameter	XAMPP	Bedeutung
Server	`"localhost"`	Die Datenbank läuft auf demselben Rechner.
Benutzer	`"root"`	Hauptbenutzer mit allen Rechten
Passwort	`""`	bei XAMPP leer
Datenbank	`"dolcevita"`	die zu verwendende Datenbank

> ⚠️ **NUR ZUM ÜBEN**
>
> `root` ohne Passwort ist bequem und nur auf dem eigenen Rechner vertretbar. Auf einem echten Server bekommt jede Website einen eigenen Datenbankbenutzer mit starkem Passwort und nur den Rechten, die sie braucht.

`set_charset("utf8mb4")` sorgt dafür, dass Umlaute und das Eurozeichen richtig zwischen PHP und Datenbank ausgetauscht werden. Ohne diese Zeile kann aus „Blutorange“ zwar nichts, aus „Kräutertee“ aber „KrÃ¤utertee“ werden.

3 | Das Ergebnis Zeile für Zeile

`query()` liefert ein **Ergebnisobjekt**. Jeder Aufruf von `fetch_assoc()` holt die nächste Zeile als assoziatives Array – mit den Spaltennamen als Schlüssel. Sind alle Zeilen abgeholt, liefert es `null`; deshalb passt es perfekt in eine `while`-Bedingung.

Abfragen und durchlaufen beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>fetch_assoc</title></head>
<body>

<?php
  $conn = new mysqli("localhost", "root", "", "dolcevita");
  $conn->set_charset("utf8mb4");

  $ergebnis = $conn->query("SELECT name, gruppe FROM sorten WHERE gruppe = 'Fruchtig' ORDER BY name");

  echo "<p>" . $ergebnis->num_rows . " fruchtige Sorten:</p>";
  echo "<ul>";
  while ($zeile = $ergebnis->fetch_assoc()) {
    echo "<li>" . $zeile["name"] . "</li>";
  }
  echo "</ul>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

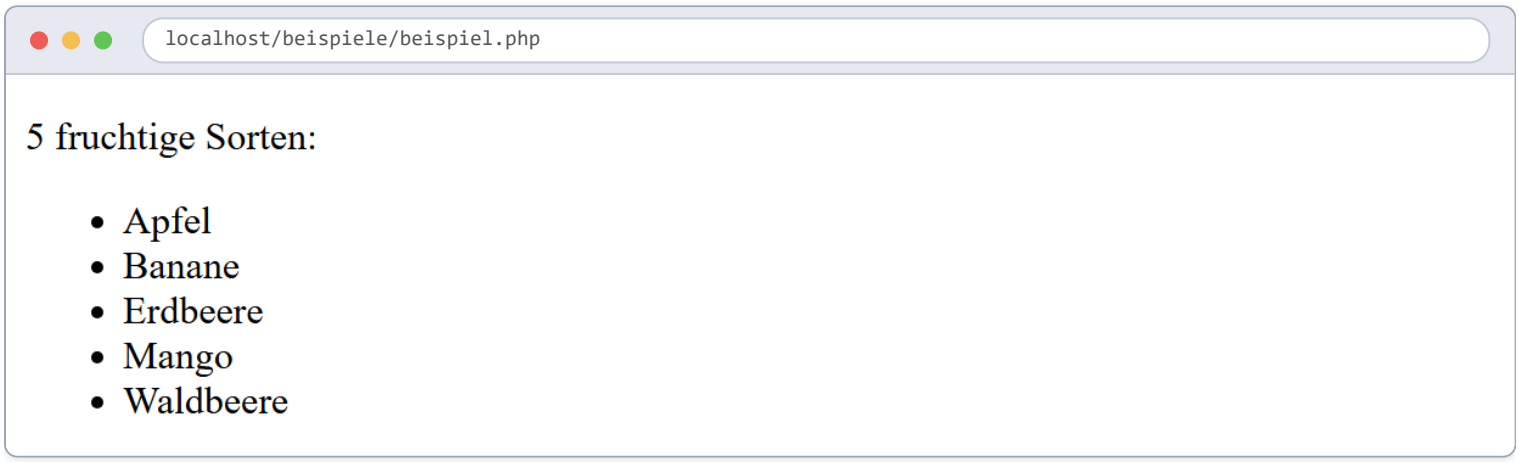


Abbildung 3: Die while-Schleife läuft genau so oft, wie es Zeilen gibt.

Ausdruck	liefert
`$conn->query($sql)`	Ergebnisobjekt (bei SELECT)
`$ergebnis->num_rows`	Anzahl der Zeilen im Ergebnis
`$ergebnis->fetch_assoc()`	nächste Zeile als Array, z. B. `["name" => "Apfel", "gruppe" => "Fruchtig"]` – oder `null`
`$ergebnis->fetch_all(MYSQLI_ASSOC)`	alle Zeilen auf einmal als Array von Datensätzen

4 | Alles auf einmal: fetch_all

`fetch_all(MYSQLI_ASSOC)` liefert genau die Form, die Sie aus LS06 kennen: ein Array von Datensätzen. Code, der mit dem Array `$sorten` arbeitet, funktioniert deshalb unverändert mit Daten aus der Datenbank.

fetch_all() beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>fetch_all</title></head>
<body>

<?php
  require_once "db_verbindung.php";

  $ergebnis = $conn->query("SELECT name, gruppe FROM sorten WHERE neu = 1");
  $neue = $ergebnis->fetch_all(MYSQLI_ASSOC);

  echo "<pre>";
  print_r($neue);
  echo "</pre>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

localhost/beispiele/beispiel.php

```
Array
(
    [0] => Array
        (
            [name] => Mango
            [gruppe] => Fruchtig
        )

    [1] => Array
        (
            [name] => Maracuja
            [gruppe] => Vegan
        )

)
```

Abbildung 4: Zwei Datensätze – dieselbe Struktur wie das Array aus LS06.

5 | Wenn etwas schiefgeht

Ist MySQL gestoppt oder das Passwort falsch, bricht PHP 8 mit einer *mysqli_sql_exception* ab:

Falsches Passwort beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Fehler</title></head>
<body>

<?php
  $conn = new mysqli("localhost", "root", "geheim", "dolcevita");
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

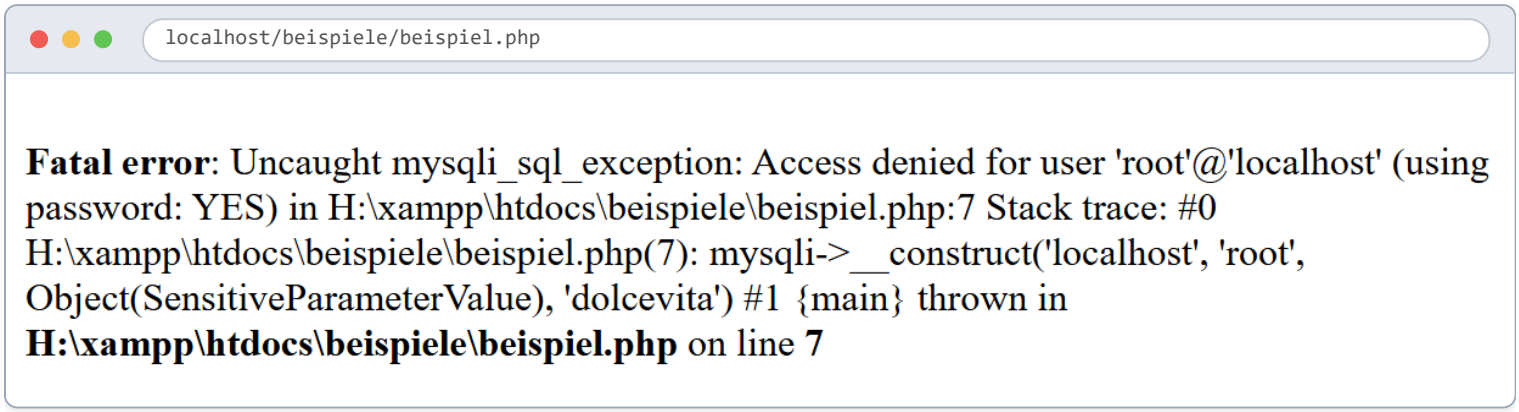


Abbildung 5: Die technische Meldung verrät Benutzername und Pfade – nichts für Besucher.

Mit `try` und `catch` fängt man den Fehler ab und zeigt stattdessen eine freundliche Meldung. So sieht `db_verbindung.php` im Projekt aus:

db_verbindung.php db_verbindung.php

```php
<?php
// ===== Verbindung zur Datenbank =====
// XAMPP-Standard: Benutzer "root" ohne Passwort. Auf einem echten Server NIE so!
$db_server   = 'localhost';
$db_benutzer = 'root';
$db_passwort = '';
$db_name     = 'dolcevita';

try {
  $conn = new mysqli($db_server, $db_benutzer, $db_passwort, $db_name);
  $conn->set_charset('utf8mb4');
} catch (mysqli_sql_exception $e) {
  die('<p>Die Datenbank ist gerade nicht erreichbar. Bitte versuchen Sie es später noch einmal.</p>');
}
```

Teil	Bedeutung
`try { … }`	„versuche“ – der Code, der schiefgehen kann
`catch (mysqli_sql_exception $e) { … }`	wird nur ausgeführt, wenn im try-Block ein Datenbankfehler auftritt
`die("…")`	gibt den Text aus und beendet das Skript